

60.2. La fonction `textePanneau()` — écrire dans le panneau

`#include <textePanneau>` : `textePanneau(colonne, ligne, "MOT")` écrit dans le panneau, comme `texte()` sur le fond.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `textePanneau(colonne, ligne, "MOT")` écrit **dans le panneau**. Les cases se comptent depuis le coin du panneau, pas depuis celui de l'écran.

La ligne à écrire : `#include <textePanneau>`. Il faut `<panneau>` pour le montrer.

Ses arguments : comme `texte()`.

Ce qu'elle coûte : environ **150 octets** la première fois.

Essaie : écris sur la ligne 1 du panneau : elle tombe sur la dernière ligne de l'écran.

On retrouve `textePanneau` dans 3 leçons

[61](#), [110.3](#), [130](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **409 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         44 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte(), textePanneau() – écrit un texte, lettre par lettre
CopierTuiles   67 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         144 o ← CopierTuiles – 9 lettres sur 43
"LE FOND"      7 o ← texte() – un texte gravé
"LE PANNEAU"   10 o ← textePanneau() – un texte gravé
```

Lettres gravées : ADEFLNOPU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
panneau()      x 1   15 o sur place, dans main()
texte()        x 1   11 o sur place, dans main() + EcrireTexte (10 o)
textePanneau() x 1   11 o sur place, dans main() + EcrireTexte (10 o)
image()        x 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

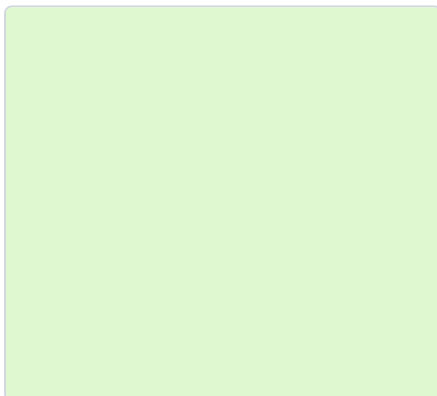
```
// ---- #include <textePanneau> : écrire dans le panneau ----  
// Le fond porte un mot ; le panneau, posé par-dessus en bas, en porte un autre.  
  
#include <texte>  
#include <textePanneau> // textePanneau() : écrire dans le panneau  
#include <panneau>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

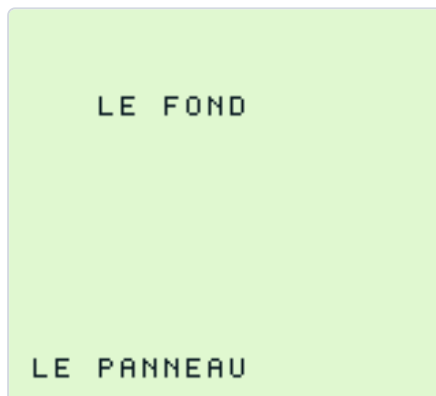
```
int main() {  
    texte(4, 4, "LE FOND");  
    textePanneau(1, 0, "LE PANNEAU"); // (1, 0) : depuis le coin du panneau  
    panneau(0, 128);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

LE FOND en haut, LE PANNEAU dans la bande du bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le fond porte son mot
- ✓ le panneau porte le sien, depuis sa case (1, 0)
- ✓ le panneau est allumé

Ensuite : **60.3. La fonction `teindrePanneau()` — une case du panneau dans une palette** — chapitre 9 : Ranger ce qu'on manipule. · [gameboy3](#) — écrire une cartouche Game Boy en C++